

03_Drag_Calculation: Coastdown-Based Estimation of CdA and Cr

Validation_Aero_Test

August 4, 2026

1 Overview

This document describes the drag and rolling-resistance estimation from extracted coastdown runs. The implementation is split between the shared library `aero_validation/drag.py` and three standalone CLI approach scripts under `Code/03_Drag_Calculation/`.

Pipeline flow

```
drag_intervals.csv + run_XXX.csv (from 02_Run_Extraction)
|
v
prepare_run_slice() --> v, v^2, decel per run
|
v
robust_linear_fit() --> k2, c per run/config
|
v
derive_drag_metrics() --> CdA, Cr, Fd
|
v
summarize_by_config() --> per-config stats + wing contributions
```

Provenance. The results shown are from the Steißlingen aero-validation test day (2025-10-22), 4 stints, 46 extracted coastdown runs across 4 aerodynamic configurations.

2 Inputs

- Intervals: `Outputs/02_Run_Extraction/csv/drag_intervals.csv` (from the run extraction step; see `02_Run_Extraction.pdf`)
- Per-run time series: `Outputs/02_Run_Extraction/csv/run_{XXX}.csv`
- Vehicle + analysis constants: `Data/vehicle_constants.json`

The following columns from the cleaned dataset (see `01_Data_Preparation.pdf`) are required in each per-run CSV:

- `t_s` [s]
- `GPS1_VEL_MAG_ms` [m s^{-1}]
- `IMU_ACCEL_LONG_ms2` [m s^{-2}]

3 Reproducibility

Run the following commands from the repository root after completing the data-preparation and run-extraction steps:

```
# Approach 1: per-run regression
python Code/03_Drag_Calculation/Approach1_PerRun/run.py

# Approach 2: paired opposite-direction averaging
python Code/03_Drag_Calculation/Approach2_PairedOpposites/run.py

# Approach 3: one pooled fit per configuration
python Code/03_Drag_Calculation/Approach3_PooledByConfig/run.py

# Optional: regenerate bootstrap uncertainty estimates
python Code/03_Drag_Calculation/bootstrap_pooled_uncertainty_from_runs.py
```

All thresholds (minimum speed, filter gates) and the vehicle constants JSON path can be overridden via CLI arguments of each approach.

4 Preprocessing per interval

Preprocessing is implemented in `aero_validation/drag.py` as `prepare_run_slice`. Given a per-run CSV and an interval $[t_0, t_1]$:

1. Slice to $t \in [t_0, t_1]$.
2. (Best effort) enforce coastdown conditions if available: throttle ICU_Apps_pct $\leq 1\%$, brakes ≤ 0.1 bar.
3. Discard low-speed samples: $v \geq v_{\min}$ (default 5 m/s).
4. Define positive deceleration as $a_{\text{dec}} = -a_{\text{long}}$.
5. Smooth a_{dec} with a centered rolling median (default window 7 samples).
6. Construct regression features v and v^2 .

5 Model, fitting, and conversions

Coastdown model

Neglecting grade and wind, the coastdown dynamics can be approximated by

$$m \dot{v} = -\frac{1}{2} \rho C_d A v^2 - C_r m g.$$

Using the positive deceleration convention $a_{\text{dec}} = -\dot{v}$, the pipeline fits

$$a_{\text{dec}}(v) = k_2 v^2 + c.$$

The conversions from regression coefficients to physical parameters are:

$$C_d A = \frac{2mk_2}{\rho}, \quad C_r = \frac{c}{g}.$$

If a frontal area A is provided per configuration, then

$$C_d = \frac{C_d A}{A}.$$

For reference, the drag and rolling-resistance forces at a given speed v are

$$F_d(v) = \frac{1}{2} \rho C_d A v^2, \quad F_{rr} = C_r m g,$$

so that $m a_{\text{dec}}(v) = F_d(v) + F_{rr}$.

Robust regression

Two robust fitting helpers (in `aero_validation/drag.py`) are used:

- `robust_linear_fit`: iterative 2σ clipping for the 1D model $y = mx + b$.
- `robust_linear_fit_multi`: iterative 2σ clipping for a multivariate linear model $y = X\beta$.

Both perform an initial least-squares fit, compute residuals, remove outliers exceeding a 2σ threshold (σ = standard deviation of residuals), and refit for up to 5 iterations. If fewer than 10 points remain after any iteration, fitting stops and the last valid coefficients are returned.

Vehicle constants and configuration mapping

Constants are loaded from `Data/vehicle_constants.json` via `load_constants` (defined in `aero_validation/drag.py`), which returns a `VehicleConstants` dataclass. Missing fields emit a `UserWarning` and use sensible defaults, except `config_ranges_by_run_id` which is required and raises a `ValueError` if absent.

Key fields:

- `vehicle.mass_kg`, `vehicle.rho_kgm3`
- `vehicle.frontal_area_m2_by_config` (optional, for reporting C_d)
- `analysis.fd_speeds_ms` (reference speeds for drag force table)
- `analysis.baseline_config` (reference configuration for $\Delta C_d A$ and wing contributions)
- `analysis.config_ranges_by_run_id` mapping config name $\rightarrow$ [start, end]

Wing contribution decomposition

The aerodynamic configurations are designed to isolate individual wing elements:

<code>full</code>	Both wings installed
<code>no_rear</code>	Rear wing removed
<code>no_front_rear</code>	Both wings removed (baseline)
<code>no_front</code>	Front wing removed

Wing contributions are computed by differencing the per-configuration mean $C_d A$ against the baseline:

$$\begin{aligned} C_d A_{\text{front_wing}} &= C_d A_{\text{no_rear}} - C_d A_{\text{baseline}}, \\ C_d A_{\text{rear_wing}} &= C_d A_{\text{no_front}} - C_d A_{\text{baseline}}, \\ C_d A_{\text{both_wings}} &= C_d A_{\text{full}} - C_d A_{\text{baseline}}. \end{aligned}$$

These are written to `drag_config_summary_contributions.csv` by each approach and reported in the results section below.

6 Implemented approaches (with example plots)

Each approach is a standalone CLI script that writes a self-contained output folder under `Outputs/03_Drag_Calculation/`.

6.1 Approach 1: Per-run (baseline)

`Code/03_Drag_Calculation/Approach1_PerRun/run.py`

- Fit $a_{\text{dec}} = k_2 v^2 + c$ per extracted interval.
- Convert k_2, c to $C_d A$ and C_r , then summarize per configuration.
- Emit per-run fit plots for QC.

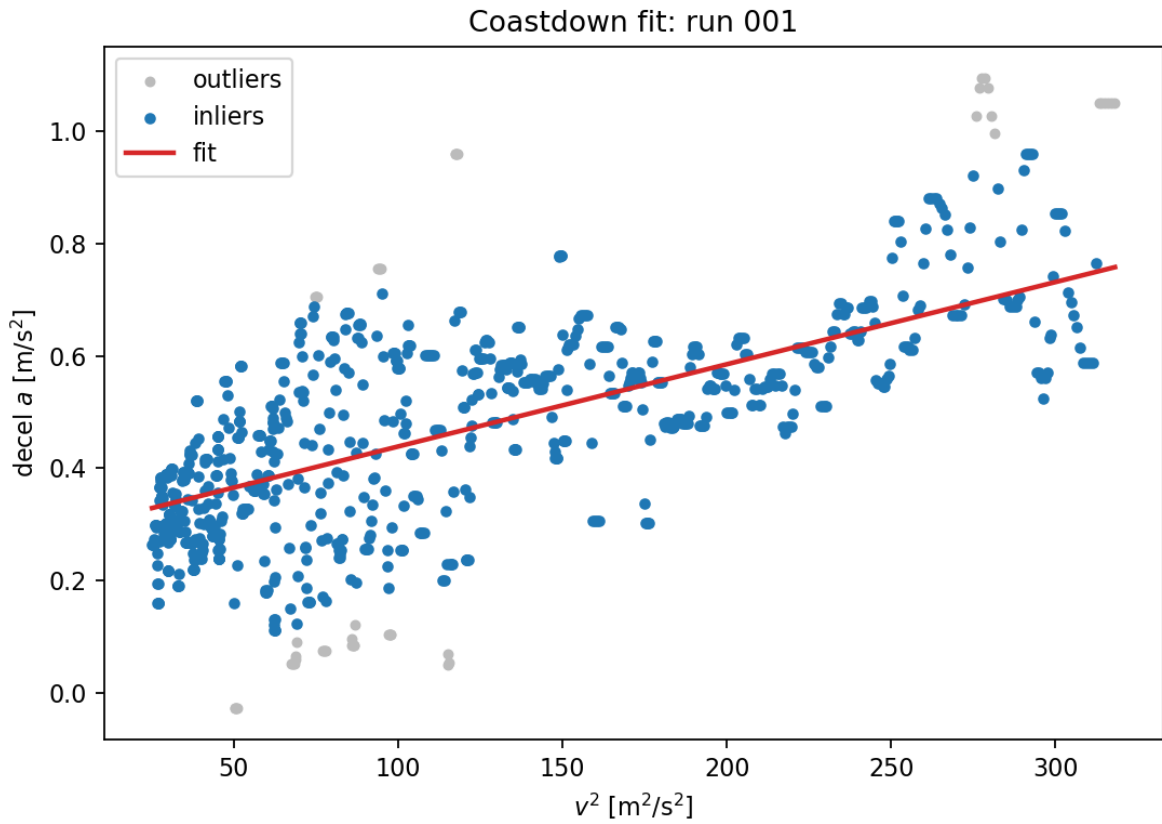


Figure 1: Approach 1: example per-run coastdown fit plot (Run 001).

6.2 Approach 2: Paired opposites (baseline)

`Code/03_Drag_Calculation/Approach2_PairedOpposites/run.py`

- Fit per run (as in Approach 1).
- Pair consecutive run IDs ($i, i+1$) within a configuration and average coefficients.
- Motivation: reduce steady biases that change sign with direction (e.g., wind/grade) when the experiment is executed as out-and-back passes.

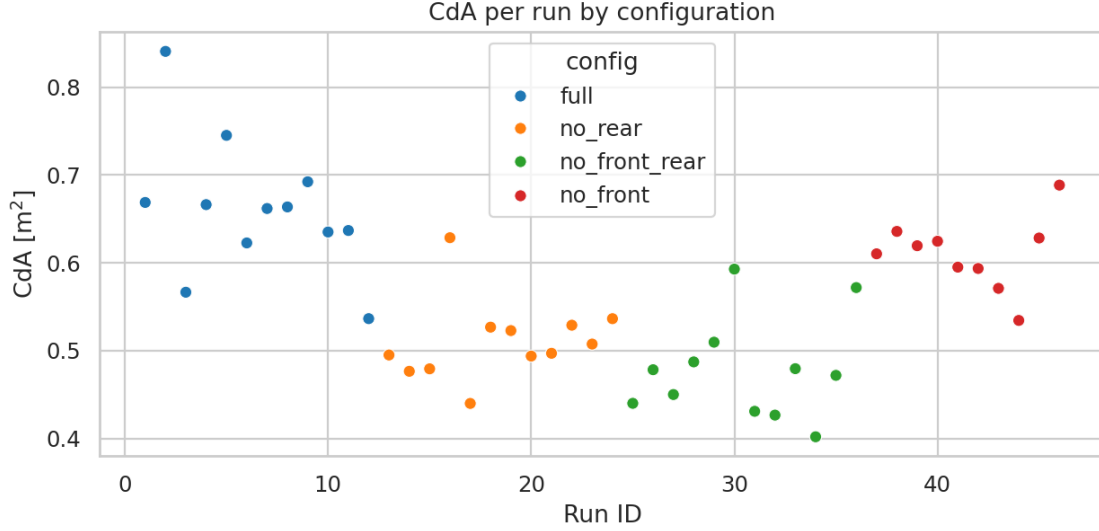


Figure 2: Approach 1: per-run C_dA values grouped by configuration.

6.3 Approach 3: Pooled by configuration (baseline)

Code/03_Drag_Calculation/Approach3_PooledByConfig/run.py

- Stack all samples of all runs belonging to a configuration.
- Fit one k_2, c per configuration using robust regression.
- Motivation: maximize point count and reduce sensitivity to short noisy runs.

7 Outputs

Each approach writes:

- Outputs/03_Drag_Calculation/<Approach>/csv/: coefficients, per-config summaries, contributions, and a drag-force table.
- Outputs/03_Drag_Calculation/<Approach>/plots/: summary plots, and (for per-run modes) per-run coastdown fit plots.

8 Results

This section summarizes the results currently present in Outputs/03_Drag_Calculation/.

8.1 Uncertainty definition

The results tables below report measurement uncertainty per configuration:

- **Approach 1:** standard deviation across runs (CdA_m2_std, Cr_std from `drag_config_summary.csv`).
- **Approach 2:** standard deviation across paired runs.
- **Approach 3:** bootstrap standard deviation of the mean across runs. The bootstrap resamples the per-run C_dA and C_r values (from Approach 1) with replacement for $n = 300$ iterations (seed = 1) and reports the standard deviation of the bootstrap means. Output is written to `bootstrap_pooled_uncertainty.csv`.

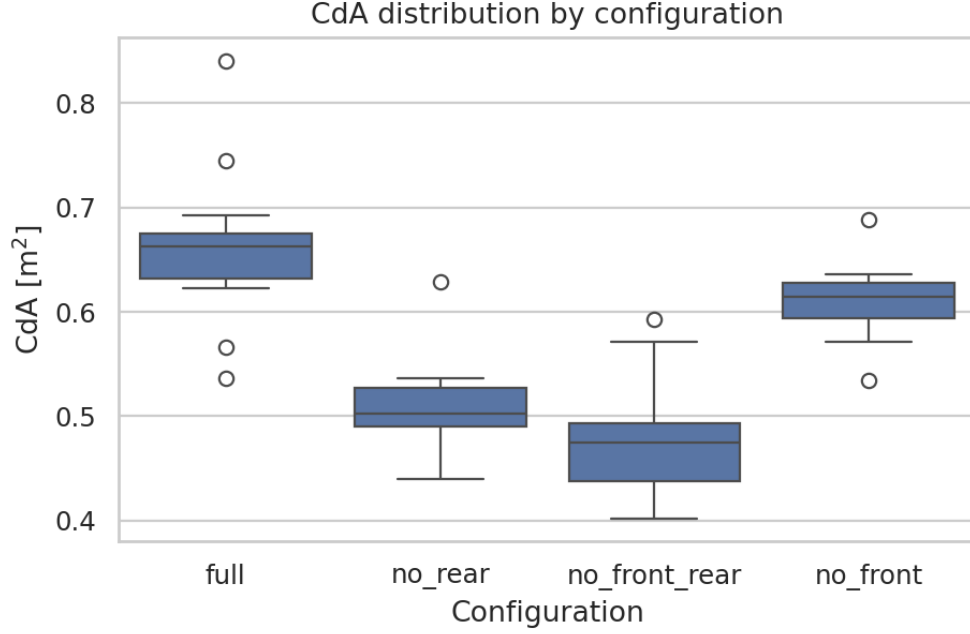


Figure 3: Approach 1: C_dA distribution per configuration (box plot).

8.2 Baseline model results (Approaches 1–3)

Table ?? summarizes the C_dA , C_r , and wing-contribution estimates across all three approaches.

Table 1: Baseline model: C_dA per configuration across Approaches 1–3 (mean/pooled value $\pm$ uncertainty).

Config	A1 (per-run)	A2 (paired)	A3 (pooled)
full	0.661 ± 0.078	0.661 ± 0.058	0.679 ± 0.022
no_rear	0.511 ± 0.046	0.511 ± 0.026	0.506 ± 0.014
no_front_rear	0.478 ± 0.057	0.478 ± 0.048	0.487 ± 0.015
no_front	0.610 ± 0.041	0.610 ± 0.039	0.647 ± 0.012

Overview: C_dA

Table 2: Baseline model: C_r per configuration across Approaches 1–3 (mean/pooled value $\pm$ uncertainty).

Config	A1 (per-run)	A2 (paired)	A3 (pooled)
full	0.0235 ± 0.0043	0.0235 ± 0.0014	0.0227 ± 0.0012
no_rear	0.0217 ± 0.0035	0.0217 ± 0.0015	0.0217 ± 0.0009
no_front_rear	0.0205 ± 0.0028	0.0205 ± 0.0013	0.0202 ± 0.0008
no_front	0.0217 ± 0.0033	0.0217 ± 0.0011	0.0201 ± 0.0010

Overview: C_r

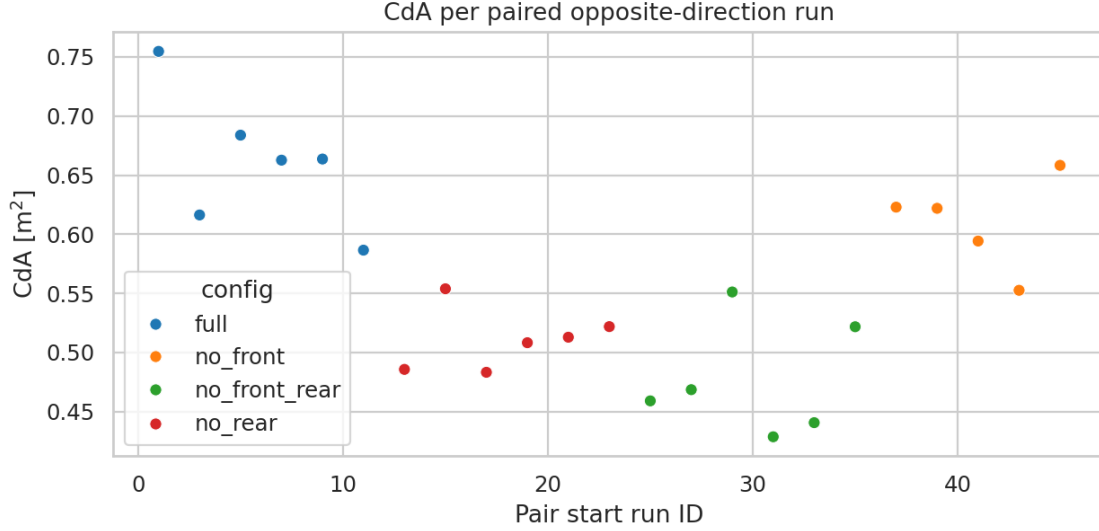


Figure 4: Approach 2: C_dA per paired runs, grouped by configuration.

9 Error handling & robustness

- **Insufficient data:** if a run slice has fewer than 10 valid points after filtering, `robust_linear_fit` raises a `ValueError` and the run is skipped.
- **Negative R^2 :** reported as-is; indicates the fit is worse than a constant model (possible if deceleration is near-constant across the speed range, e.g., on a very short run).
- **Empty configuration:** if a config has zero valid runs, `summarize_by_config` raises a `ValueError` when the baseline is missing; Approach 3 skips configs with < 50 pooled samples.
- **Unknown run ID:** runs outside all configured ranges are assigned the label "unknown" and excluded from per-config summaries.

10 Notes and limitations

- Grade and wind are not explicitly modeled inside a single run; pairing opposite directions reduces steady biases if the experiment is designed as out-and-back passes.
- The F_{rr} force is provided for reference but is constant per configuration and does not vary with speed; the primary aerodynamic quantity of interest is C_dA .
- All thresholds (minimum speed, filter gates, bootstrap iterations) can be adjusted via CLI arguments of each approach.

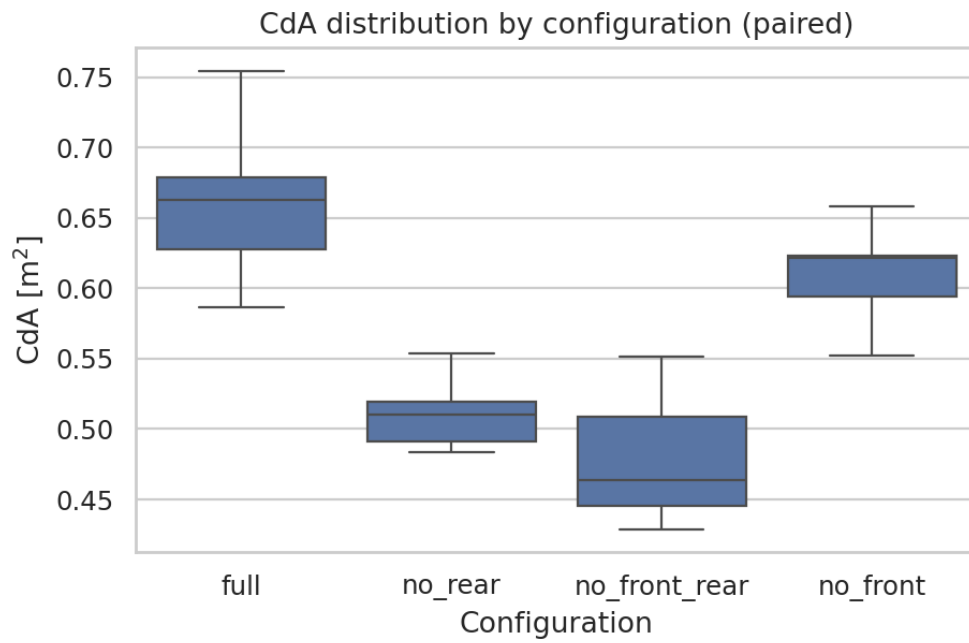


Figure 5: Approach 2: C_dA distribution per configuration after pairing.

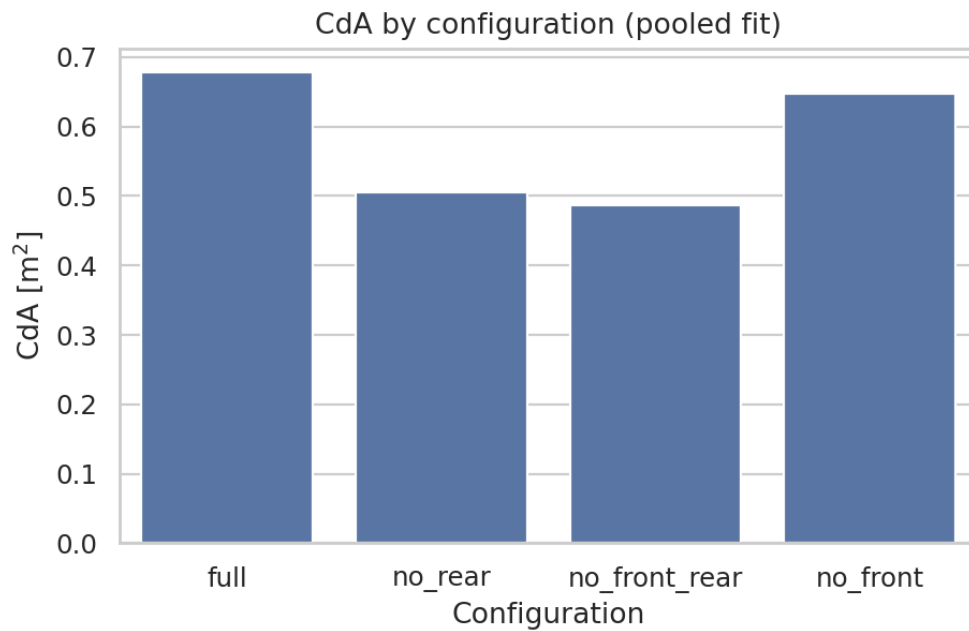


Figure 6: Approach 3: pooled C_dA estimate per configuration.